
Corrigé

Architecture des ordinateurs

Architecture et fonctionnement d'un ordinateur,
systèmes de numérotation et représentation de l'information

Chapitres :

1. Architecture et fonctionnement d'un ordinateur
2. Systèmes de numérotation et représentation de l'information

Année universitaire 2025–2026

1 Architecture et fonctionnement d'un ordinateur

Correction – Exercice 1.1 : QCM Généralités sur l'architecture

-) **CPU** : réponses **i** et **iii**. Le CPU est bien le cerveau de l'ordinateur et est composé de l'UAL (unité arithmétique et logique) et de l'UC (unité de contrôle). Il n'exécute pas uniquement des opérations arithmétiques (il gère aussi les opérations logiques et le contrôle) et ne stocke pas les données permanentes (c'est le rôle du disque dur).
-) **Architecture de Von Neumann** : réponses **ii** et **iii**. Le principe fondamental de Von Neumann est que programme et données sont stockés dans la même mémoire. Les bus relient les composants. La réponse i est fausse : c'est justement le contraire, Von Neumann ne sépare pas les données des instructions (contrairement à Harvard). La réponse iv est fausse : l'unité de contrôle est bien présente.
-) **Mémoire RAM** : réponses **ii**, **iii** et **iv**. La RAM est volatile (elle perd ses données sans alimentation), plus rapide que le disque dur, et stocke les programmes en cours d'exécution. La réponse i est fausse : la RAM est volatile, elle ne conserve pas ses données hors tension.
-) **Bus** : réponses **i**, **ii** et **iv**. Un bus est un circuit de communication, il transporte adresses et données, et le nombre de lignes du bus d'adresses détermine la capacité mémoire adressable. La réponse iii est fausse : les bus de données et d'adresses sont bidirectionnels.

Correction – Exercice 1.2 : Identification des composants

Composant	Fonction principale
Processeur (CPU)	Exécuter les instructions des programmes ; effectuer les calculs et la logique
Mémoire vive (RAM)	Stocker temporairement les données et programmes en cours d'utilisation (mémoire volatile)
Disque dur / SSD	Stocker les données de manière permanente (mémoire non volatile)
Carte mère	Relier et communiquer tous les composants ; accueillir le CPU, la RAM, les cartes d'extension
Carte graphique (GPU)	Traiter et afficher les images ; accélérer les calculs graphiques et parallèles
Alimentation	Convertir le courant électrique et fournir l'énergie à tous les composants

Correction – Exercice 1.3 : Modèle de Von Neumann

1.) Schéma simplifié du modèle de Von Neumann :

Unité de contrôle (UC)

Entrées/Sorties

Les quatre composants fondamentaux sont : l'**unité de contrôle**, l'**unité arithmétique et logique** (UAL), la **mémoire centrale** et les **unités d'entrée/sortie**. Ils sont reliés par des **bus** (bus de données, bus d'adresses, bus de commande).

2.) Cycle d'instruction (fetch-decode-execute) :

- **Phase de recherche (Fetch)** : l'UC va chercher l'instruction en mémoire à l'adresse indiquée par le compteur ordinal (PC), puis la charge dans le registre d'instruction (IR). Le PC est incrémenté.

- **Phase de décodage (Decode)** : l'UC décode l'instruction pour déterminer l'opération à effectuer et les opérandes nécessaires.
- **Phase d'exécution (Execute)** : l'UAL effectue l'opération, le résultat est stocké, et les éventuels transferts mémoire sont réalisés.

3.) **Différence Von Neumann / Harvard** : Dans l'architecture de **Von Neumann**, programme et données partagent la même mémoire et les mêmes bus. Dans l'architecture **Harvard**, il y a des mémoires séparées et des bus distincts pour les instructions et les données, ce qui permet un accès simultané et donc de meilleures performances.

Correction – Exercice 1.4 : Cycle d'instruction

Soit l'instruction `ADD R1, R2, R3` ($R1 \leftarrow R2 + R3$).

1.) **Les trois phases :**

- **Fetch** : l'UC va chercher l'instruction `ADD R1, R2, R3` en mémoire à l'adresse contenue dans PC.
- **Decode** : l'UC identifie l'opération (addition), les registres sources (R2, R3) et le registre destination (R1).
- **Execute** : l'UAL lit le contenu de R2 et R3, effectue l'addition, et le résultat est écrit dans R1.

2.) **Composants sollicités :**

- **Fetch** : PC (compteur ordinal), mémoire centrale, bus d'adresses et de données, IR (registre d'instruction).
- **Decode** : UC (unité de contrôle), décodeur d'instructions.
- **Execute** : UAL, registres R1, R2, R3.

3.) **Rôle du PC et de l'IR :**

- Le **PC (Program Counter)** contient l'adresse mémoire de la prochaine instruction à exécuter. Il est incrémenté automatiquement après chaque fetch.
- L'**IR (Instruction Register)** contient l'instruction courante qui vient d'être chargée depuis la mémoire, en attente d'être décodée puis exécutée.

Correction – Exercice 1.5 : Hiérarchie mémoire

1.) **Du plus rapide au plus lent** : Registres > Cache L1 > Cache L2 > Mémoire RAM > Disque dur SSD > Disque dur HDD

2.) **De la plus petite à la plus grande capacité** : Registres (octets) < Cache L1 (Ko) < Cache L2 (Mo) < Mémoire RAM (Go) < Disque dur SSD (Go/To) < Disque dur HDD (Go/To)

3.) **Principes de localité :**

- **Localité temporelle** : si une donnée est accédée, il est probable qu'elle sera accédée à nouveau dans un futur proche. Le cache conserve les données récemment utilisées.
- **Localité spatiale** : si une donnée est accédée, il est probable que les données voisines en mémoire seront aussi accédées. Le cache charge des blocs de données contiguës (lignes de cache).

4.) **Nécessité de la hiérarchie** : il existe un compromis entre vitesse, capacité et coût. Les mémoires rapides sont petites et chères, les mémoires grandes sont lentes et bon marché. La

hiérarchie permet de donner l'illusion d'une mémoire à la fois grande et rapide en exploitant les principes de localité.

Remarque

La mémoire cache fonctionne de manière transparente pour le programmeur : quand le CPU cherche une donnée, il consulte d'abord le cache L1, puis L2, puis la RAM. Si la donnée est dans le cache, on parle de **cache hit** ; sinon, c'est un **cache miss**, et la donnée est chargée depuis le niveau inférieur.

Correction – Exercice 1.6 : Bus et interconnexions

1.) **Mémoire adressable avec 32 bits** : $2^{32} = 4\,294\,967\,296$ octets = 4 Gio $\approx 4,29$ Go.

2.) **Bus d'adresses de 24 bits** : $2^{24} = 16\,777\,216$ octets = 16 Mio $\approx 16,78$ Mo.

3.) **Différence entre les trois types de bus** :

- **Bus de données** : transporte les données entre le CPU, la mémoire et les périphériques. Sa largeur (en bits) détermine la quantité de données transmise en une seule opération.
- **Bus d'adresses** : transporte les adresses mémoire générées par le CPU pour désigner la case mémoire ou le périphérique à accéder. Sa largeur détermine l'espace adressable.
- **Bus de commande** : transporte les signaux de contrôle (lecture/écriture, horloge, interruptions, etc.) pour synchroniser et piloter les opérations.

Correction – Exercice 1.7 : Entrées / Sorties

1.) **Classification des périphériques** :

- **Entrée** : clavier, souris, microphone, webcam, scanner
- **Sortie** : écran, imprimante, haut-parleurs
- **Entrée/Sortie** : disque dur, modem

2.) **Contrôleur de périphérique** : c'est un circuit électronique qui fait l'interface entre le processeur/bus et un périphérique. Il reçoit les commandes du CPU, les traduit en signaux compréhensibles par le périphérique, gère les transferts de données et signale la fin des opérations (par interruption ou polling).

3.) **Port série vs port parallèle** :

- **Port série** : transmet les données bit par bit sur un seul fil. Plus simple, moins coûteux, adapté aux longues distances. Exemple : USB, port série RS-232.
- **Port parallèle** : transmet plusieurs bits simultanément sur plusieurs fils. Plus rapide sur de courtes distances mais sujet aux interférences et au coût de câblage. Exemple : ancien port parallèle d'imprimante (IEEE 1284), bus PCI.

Correction – Exercice 1.8 : Jeu d'instructions

Séquence d'instructions calculant $A + B - C$:

- 1.) `LOAD R1, @A` // $R1 \leftarrow A$
- 2.) `LOAD R2, @B` // $R2 \leftarrow B$
- 3.) `LOAD R3, @C` // $R3 \leftarrow C$
- 4.) `ADD R1, R2` // $R1 \leftarrow R1 + R2 = A + B$

5.) SUB R1, R3 // $R1 \leftarrow R1 - R3 = A + B - C$

6.) STORE R1, @R // $@R \leftarrow R1 = A + B - C$

Remarque

Cette séquence illustre le principe de l'architecture load/store : les opérations arithmétiques ne portent que sur des registres, et les transferts mémoire se font exclusivement via les instructions LOAD et STORE. Ce modèle est utilisé dans la plupart des processeurs RISC modernes (ARM, MIPS, RISC-V).

Correction – Exercice 1.9 : Logiciel et système d'exploitation

1.) Logiciel système vs logiciel applicatif :

- **Logiciel système** : programmes qui gèrent le fonctionnement de l'ordinateur. Exemples : système d'exploitation (Windows, Linux), pilotes (drivers), utilitaires de disque.
- **Logiciel applicatif** : programmes conçus pour accomplir des tâches pour l'utilisateur. Exemples : traitement de texte (Word), navigateur web (Firefox), tableur (Excel).

2.) Rôles principaux du système d'exploitation :

- Gestion des processus (ordonnancement, création, terminaison)
- Gestion de la mémoire (allocation, protection, mémoire virtuelle)
- Gestion des fichiers (système de fichiers, droits d'accès)
- Gestion des périphériques (pilotes, interruptions)
- Interface utilisateur (CLI ou GUI)
- Sécurité et protection (authentification, isolation)

3.) Compilateur, interpréteur, assembleur :

- **Compilateur** : traduit l'intégralité du code source en code machine (fichier exécutable) avant l'exécution. Exemples : gcc, javac.
- **Interpréteur** : traduit et exécute le code source ligne par ligne, sans produire de fichier exécutable. Exemples : Python, PHP.
- **Assembleur** : traduit le langage assembleur (mnémoniques) en langage machine (binaire), avec une correspondance quasi 1-pour-1.

Correction – Exercice 1.10 : Tableau récapitulatif des unités

Valeur en octets	Abréviation	Signification
$2^{10} = 1024$	Kio	Kibiocet (kilo binaire)
$2^{20} = 1\,048\,576$	Mio	Mébiocet (méga binaire)
$2^{30} = 1\,073\,741\,824$	Gio	Gibiocet (giga binaire)
$2^{40} = 1\,099\,511\,627\,776$	Tio	Tébiocet (téra binaire)
$10^3 = 1000$	Ko	Kilooctet (décimal)
$10^6 = 1\,000\,000$	Mo	Mégaoctet (décimal)
$10^9 = 1\,000\,000\,000$	Go	Gigaoctet (décimal)

2 Systèmes de numérotation et représentation de l'information

Correction – Exercice 2.1 : QCM Systèmes de numérotation

-) **Système décimal** : réponse **iii** (base 10).
-) **Chiffres hexadécimaux** : réponse **ii** (0 à 9 et A à F, soit 16 symboles). La réponse iii est fausse car les chiffres au-delà de 9 sont représentés par des lettres, et la réponse iv est incorrecte car seules les lettres A à F sont utilisées.
-) $(1010)_2$ **en décimal** : réponse **ii** ($1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 8 + 0 + 2 + 0 = 10$).
-) **Bit de poids fort** : réponses **ii** et **iv**. Le bit de poids fort (MSB – Most Significant Bit) est le bit le plus à gauche, et en complément à deux, il détermine le signe du nombre.

Correction – Exercice 2.2 : Conversions décimal $\rightarrow$ binaire

- 1.) $25 = 11001_2$ ($25 \div 2 = 12$ r1, $12 \div 2 = 6$ r0, $6 \div 2 = 3$ r0, $3 \div 2 = 1$ r1, $1 \div 2 = 0$ r1)
- 2.) $47 = 101111_2$ ($47 \div 2 = 23$ r1, $23 \div 2 = 11$ r1, $11 \div 2 = 5$ r1, $5 \div 2 = 2$ r1, $2 \div 2 = 1$ r0, $1 \div 2 = 0$ r1)
- 3.) $128 = 1000000_2$ ($128 = 2^7$)
- 4.) $200 = 11001000_2$ ($200 \div 2 = 100$ r0, $100 \div 2 = 50$ r0, $50 \div 2 = 25$ r0, $25 \div 2 = 12$ r1, $12 \div 2 = 6$ r0, $6 \div 2 = 3$ r0, $3 \div 2 = 1$ r1, $1 \div 2 = 0$ r1)
- 5.) $255 = 1111111_2$ ($255 = 2^8 - 1$)
- 6.) $1024 = 1000000000_2$ ($1024 = 2^{10}$)

Correction – Exercice 2.3 : Conversions binaire $\rightarrow$ décimal

- 1.) $(1101)_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 4 + 0 + 1 = \mathbf{13}$
- 2.) $(10110)_2 = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 16 + 0 + 4 + 2 + 0 = \mathbf{22}$
- 3.) $(11111111)_2 = 2^8 - 1 = \mathbf{255}$
- 4.) $(10000000)_2 = 2^7 = \mathbf{128}$
- 5.) $(1100100)_2 = 2^6 + 2^5 + 2^2 = 64 + 32 + 4 = \mathbf{100}$
- 6.) $(101010101)_2 = 2^8 + 2^6 + 2^4 + 2^2 + 2^0 = 256 + 64 + 16 + 4 + 1 = \mathbf{341}$

Correction – Exercice 2.4 : Conversions décimal $\rightarrow$ hexadécimal

- 1.) $255 = \mathbf{FF}_{16}$ ($255 \div 16 = 15$ r15; $15 = \mathbf{F}$)
- 2.) $512 = \mathbf{200}_{16}$ ($512 \div 16 = 32$ r0; $32 \div 16 = 2$ r0; $2 \div 16 = 0$ r2)
- 3.) $1024 = \mathbf{400}_{16}$ ($1024 \div 16 = 64$ r0; $64 \div 16 = 4$ r0; $4 \div 16 = 0$ r4)
- 4.) $4096 = \mathbf{1000}_{16}$ ($4096 = 16^3$)
- 5.) $65535 = \mathbf{FFFF}_{16}$ ($65535 = 16^4 - 1$)
- 6.) $1000 = \mathbf{3E8}_{16}$ ($1000 \div 16 = 62$ r8; $62 \div 16 = 3$ r14 = E; $3 \div 16 = 0$ r3)

Correction – Exercice 2.5 : Conversions hexadécimal → décimal

- 1.) $(1A)_{16} = 1 \times 16^1 + 10 \times 16^0 = 16 + 10 = \mathbf{26}$
- 2.) $(FF)_{16} = 15 \times 16^1 + 15 \times 16^0 = 240 + 15 = \mathbf{255}$
- 3.) $(100)_{16} = 1 \times 16^2 = \mathbf{256}$
- 4.) $(ABC)_{16} = 10 \times 16^2 + 11 \times 16^1 + 12 \times 16^0 = 2560 + 176 + 12 = \mathbf{2748}$
- 5.) $(FFFF)_{16} = 15 \times 16^3 + 15 \times 16^2 + 15 \times 16^1 + 15 \times 16^0 = 61440 + 3840 + 240 + 15 = \mathbf{65535}$
- 6.) $(3E8)_{16} = 3 \times 16^2 + 14 \times 16^1 + 8 \times 16^0 = 768 + 224 + 8 = \mathbf{1000}$

Correction – Exercice 2.6 : Conversions binaire ↔ hexadécimal

- 1.) **Binaire → Hexadécimal** (regroupement par blocs de 4 bits) :
 - $(11010110)_2 = D6_{16}$ car 1101 = D et 0110 = 6
 - $(101111001010)_2 = BCA_{16}$ car 1011 = B, 1100 = C, 1010 = A
 - $(111100001111)_2 = F0F_{16}$ car 1111 = F, 0000 = 0, 1111 = F
- 2.) **Hexadécimal → Binaire** (chaque chiffre hexa → 4 bits) :
 - $(3F)_{16} = (00111111)_2$ car 3 = 0011 et F = 1111
 - $(A5)_{16} = (10100101)_2$ car A = 1010 et 5 = 0101
 - $(C0FE)_{16} = (1100000011111110)_2$ car C = 1100, 0 = 0000, F = 1111, E = 1110

Remarque

La conversion binaire ↔ hexadécimal est très simple car $16 = 2^4$. Chaque chiffre hexadécimal correspond exactement à 4 bits. C'est pourquoi l'hexadécimal est très utilisé en informatique : il permet de représenter compactement des valeurs binaires tout en facilitant la conversion.

Correction – Exercice 2.7 : Conversions octal ↔ décimal

- 1.) **Octal → Décimal** :
 - $(37)_8 = 3 \times 8^1 + 7 \times 8^0 = 24 + 7 = \mathbf{31}$
 - $(144)_8 = 1 \times 8^2 + 4 \times 8^1 + 4 \times 8^0 = 64 + 32 + 4 = \mathbf{100}$
 - $(777)_8 = 7 \times 8^2 + 7 \times 8^1 + 7 \times 8^0 = 448 + 56 + 7 = \mathbf{511}$
- 2.) **Décimal → Octal** :
 - $64 = \mathbf{100}_8$ ($64 \div 8 = 8$ r0, $8 \div 8 = 1$ r0, $1 \div 8 = 0$ r1)
 - $255 = \mathbf{377}_8$ ($255 \div 8 = 31$ r7, $31 \div 8 = 3$ r7, $3 \div 8 = 0$ r3)
 - $1024 = \mathbf{2000}_8$ ($1024 \div 8 = 128$ r0, $128 \div 8 = 16$ r0, $16 \div 8 = 2$ r0, $2 \div 8 = 0$ r2)

Correction – Exercice 2.8 : Opérations en binaire

- 1.) $(1101)_2 + (1011)_2 = \mathbf{11000}_2$

$$\begin{array}{r}
 1\ 1\ 0\ 1 \\
 +\ 1\ 0\ 1\ 1 \\
 \hline
 1\ 1\ 0\ 0\ 0
 \end{array}$$

Retenues : de la droite vers la gauche : $1+1=10$, $0+1+1=10$, $1+0+1=10$, $1+1+1=11$

2.) $(10110)_2 + (01101)_2 = \mathbf{100011}_2$

$$\begin{array}{r} 1\ 0\ 1\ 1\ 0 \\ +0\ 1\ 1\ 0\ 1 \\ \hline 1\ 0\ 0\ 0\ 1\ 1 \end{array}$$

3.) $(11001)_2 - (10010)_2 = \mathbf{0111}_2 = \mathbf{111}_2$

$$\begin{array}{r} 1\ 1\ 0\ 0\ 1 \\ -1\ 0\ 0\ 1\ 0 \\ \hline 1\ 1\ 1 \end{array}$$

4.) $(10110)_2 - (01011)_2 = \mathbf{01011}_2 = \mathbf{1011}_2$

$$\begin{array}{r} 1\ 0\ 1\ 1\ 0 \\ -0\ 1\ 0\ 1\ 1 \\ \hline 0\ 1\ 0\ 1\ 1 \end{array}$$

5.) $(1101)_2 \times (11)_2 = \mathbf{100111}_2$

$$\begin{array}{r} 1\ 1\ 0\ 1 \\ \times \quad 1\ 1 \\ \hline 1\ 1\ 0\ 1 \\ 1\ 1\ 0\ 1\ . \\ \hline 1\ 0\ 0\ 1\ 1\ 1 \end{array}$$

Vérification : $13 \times 3 = 39$; $(100111)_2 = 32 + 4 + 2 + 1 = 39$

6.) $(11010)_2 \div (110)_2 = \mathbf{100}_2$ quotient, reste $\mathbf{10}_2$ Vérification : $26 \div 6 = 4$ reste 2

Correction – Exercice 2.9 : Représentation des entiers signés

25 en binaire sur 8 bits : **00011001**

1.) **Signe et valeur absolue :**

- +25 : 0 0011001 (bit de signe 0 + valeur absolue)
- -25 : 1 0011001 (bit de signe 1 + valeur absolue)

2.) **Complément à un :**

- +25 : 00011001 (identique à la représentation normale)
- -25 : 11100110 (inversion de tous les bits de +25)

3.) Complément à deux :

- +25 : 00011001
- -25 : 11100111 (complément à un +1 : 11100110 + 1 = 11100111)

4.) +100 et -100 en complément à deux :

- 100 = 01100100₂
- +100 : 01100100
- -100 : complément à un = 10011011; +1 = 10011100

5.) Intervalle sur 8 bits en complément à deux :

$$-2^7 \leq N \leq 2^7 - 1 \quad \text{soit} \quad -128 \leq N \leq +127$$

- 6.) **Avantage du complément à deux** : l'addition de deux nombres signés se fait de la même manière que pour les nombres non signés, sans traitement séparé du signe. Il n'y a qu'un seul zéro (contrairement au complément à un et à signe+ valeur absolue qui ont +0 et -0). La soustraction se ramène à une addition : $A - B = A + (-B)$.

Remarque

En complément à deux, le bit de poids fort (MSB) indique le signe : 0 pour positif, 1 pour négatif. Pour retrouver la valeur absolue d'un nombre négatif, il suffit d'appliquer à nouveau le complément à deux (inverser les bits et ajouter 1). Par exemple, $11100111 \rightarrow 00011000 + 1 = 00011001 = 25$.

Correction – Exercice 2.10 : Détection de débordement

Intervalle en complément à deux sur 8 bits : $[-128, +127]$.

- 1.) $100 + 27 = 127$: **pas de débordement** (127 est dans l'intervalle).
- 2.) $100 + 50 = 150$: **débordement** ($150 > 127$). Le résultat 150 n'est pas représentable.
- 3.) $-100 + (-50) = -150$: **débordement** ($-150 < -128$). Le résultat n'est pas représentable.
- 4.) $-100 + 27 = -73$: **pas de débordement** ($-73 \in [-128, 127]$).
- 5.) $80 + 70 = 150$: **débordement** ($150 > 127$).
- 6.) $-120 + (-20) = -140$: **débordement** ($-140 < -128$).

Règle de détection : un débordement se produit lorsque deux nombres de même signe donnent un résultat de signe opposé, ou lorsque le résultat sort de l'intervalle $[-2^{n-1}, 2^{n-1} - 1]$.

Correction – Exercice 2.11 : Représentation en virgule flottante

1.) 13.75 en binaire :

- Partie entière : $13 = 8 + 4 + 1 = 1101_2$
- Partie fractionnaire : $0.75 = 0.5 + 0.25 = 0.11_2$
- $13.75 = 1101.11_2$

2.) IEEE 754 simple précision de 13.75 :

- Normalisation : $1101.11_2 = 1.10111 \times 2^3$
- Signe $S = 0$ (positif)

- Exposant : $E = 3 + 127 = 130 = 10000010_2$
- Mantisse : $M = 10111000 \dots$ (23 bits, le 1 implicite est omis)
- Résultat : **0 10000010 10111000000000000000000**

3.) **IEEE 754 simple précision de -6.5 :**

- 6.5 en binaire : $6 = 110_2$, $0.5 = 0.1_2$, donc $6.5 = 110.1_2$
- Normalisation : $110.1_2 = 1.101 \times 2^2$
- Signe $S = 1$ (négatif)
- Exposant : $E = 2 + 127 = 129 = 10000001_2$
- Mantisse : $M = 10100 \dots$ (23 bits)
- Résultat : **1 10000001 10100000000000000000000**

4.) **Décodage de 0 10000010 01100000000000000000000 :**

- Signe : $S = 0 \Rightarrow$ positif
- Exposant : $E = 10000010_2 = 130$; exposant réel = $130 - 127 = 3$
- Mantisse : $M = 011$; valeur = $1.011_2 = 1 + 0.25 + 0.125 = 1.375$
- Valeur : $+1.375 \times 2^3 = 1.375 \times 8 = 11$

Remarque

Le format IEEE 754 possède des valeurs spéciales : l'exposant tout à 1 avec mantisse nulle représente $\pm\infty$, et l'exposant tout à 1 avec mantisse non nulle représente NaN (Not a Number). L'exposant tout à 0 avec mantisse nulle représente ± 0 .

Correction – Exercice 2.12 : Codes de caractères

1.) **Code ASCII :**

- A : 65 en décimal, 41 en hexadécimal
- a : 97 en décimal, 61 en hexadécimal

2.) **Différence :** $97 - 65 = 32$ en décimal (20 en hexadécimal). On constate que la différence entre une lettre minuscule et sa majuscule correspondante est toujours de 32, ce qui revient à inverser le 6^e bit en binaire.

3.) **Hello en ASCII hexadécimal :**

- H = 48, e = 65, l = 6C, l = 6C, o = 6F
- Hello = **48 65 6C 6C 6F**₁₆

4.) **Limite de l'ASCII et solution Unicode :** L'ASCII est limité à 128 caractères (7 bits), ce qui ne suffit que pour l'alphabet latin, les chiffres et quelques symboles. Il ne peut pas représenter les caractères des autres alphabets (arabe, chinois, japonais, etc.). **Unicode** résout ce problème en attribuant un numéro unique à chaque caractère de toutes les langues du monde, sur un espace de plus d'un million de codes possibles.

5.) **Nombre de caractères :**

- ASCII sur 7 bits : $2^7 = 128$ caractères
- Unicode UTF-8 : de 1 à 4 octets par caractère, soit potentiellement plus de $2^{21} = 2\,097\,152$ caractères différents (l'espace Unicode complet va jusqu'à 2^{20} points de code dans les plans standards, plus les plans supplémentaires)

Correction – Exercice 2.13 : Opérations logiques

1.) **AND** : $(11001010)_2$ AND $(10110011)_2$

```

  1 1 0 0 1 0 1 0
  1 0 1 1 0 0 1 1
  -----
  1 0 0 0 0 0 1 0

```

Résultat : **10000010**₂2.) **OR** : $(11001010)_2$ OR $(10110011)_2$

```

  1 1 0 0 1 0 1 0
  1 0 1 1 0 0 1 1
  -----
  1 1 1 1 1 0 1 1

```

Résultat : **11111011**₂3.) **XOR** : $(11001010)_2$ XOR $(10110011)_2$

```

  1 1 0 0 1 0 1 0
  1 0 1 1 0 0 1 1
  -----
  0 1 1 1 1 0 0 1

```

Résultat : **01111001**₂4.) **NOT** : NOT $(11001010)_2$ = **00110101**₂ (inversion bit à bit)5.) **Masquage** : $(10101100)_2$ AND $(11110000)_2$

```

  1 0 1 0 1 1 0 0
  1 1 1 1 0 0 0 0
  -----
  1 0 1 0 0 0 0 0

```

Résultat : **10100000**₂. Le masque $(11110000)_2$ a conservé les 4 bits de poids fort et mis à zéro les 4 bits de poids faible.

Remarque

Le **masquage** est une technique fondamentale en informatique. L'opération AND avec un masque permet de **mettre à 0** certains bits (ceux où le masque vaut 0) tout en conservant les autres. L'opération OR avec un masque permet de **mettre à 1** certains bits. L'opération XOR permet d'**inverser** certains bits. Ces opérations sont utilisées dans la gestion des registres de contrôle, les protocoles réseau et le traitement d'images.

Correction – Exercice 2.14 : Tableau de conversions croisés

Décimal	Binaire	Octal	Hexadécimal	
42	101010	52	2A	
85	1010101	125	55	
127	1111111	177	7F	
255	11111111	377	FF	
100	1100100	144	64	
240	11110000	360	F0	
255	11111111	377	FF	
51966	1100101011111110	145276	CAFE	

Détails des conversions :

- $42 = 32 + 8 + 2 = 101010_2$; $52_8 = 5 \times 8 + 2 = 42$; $2A_{16} = 2 \times 16 + 10 = 42$
- $(1010101)_2 = 64 + 16 + 4 + 1 = 85$
- $(177)_8 = 1 \times 64 + 7 \times 8 + 7 = 127$
- $FF_{16} = 15 \times 16 + 15 = 255$
- $(CAFE)_{16} = 12 \times 4096 + 10 \times 256 + 15 \times 16 + 14 = 49152 + 2560 + 240 + 14 = 51966$